

Chương này tập trung trình bày các nền tảng kiến thức cốt lõi về Trí tuệ Nhân tạo (AI) và các kỹ thuật tiên tiến được áp dụng để xây dựng hệ thống Trợ lý ảo giáo dục. Các khái niệm được triển khai từ mức cơ bản đến chuyên sâu, bám sát vào những công nghệ thực tế đang được sử dụng trong dự án.

0.1 Kiến thức nền tảng về Trí tuệ Nhân tạo và Xử lý ngôn ngữ tự nhiên

Khái quát về kiến trúc cốt lõi định hình nên các mô hình ngôn ngữ hiện đại.

0.1.1 Tokenization (Mã hóa từ vựng)

Quá trình chuyển đổi văn bản thô thành các đơn vị cơ sở (token) để máy tính hiểu được. Một trong những thuật toán phổ biến nhất hiện nay là **Byte-Pair Encoding (BPE)**. BPE bắt đầu bằng việc coi mỗi ký tự riêng lẻ là một token, sau đó lặp lại nhiều lần quá trình tìm cặp token xuất hiện cạnh nhau nhiều nhất và gộp chúng thành một token mới. Nguyên lý tần suất của BPE có thể biểu diễn như sau:

$$freq(t_a, t_b) = \sum_{w \in D} C(w) \cdot count((t_a, t_b), w) \quad (1)$$

Trong đó:

- D : Tập dữ liệu văn bản đào tạo.
- $C(w)$: Số lần xuất hiện của từ w trong D .
- $count((t_a, t_b), w)$: Số lượng cặp token (t_a, t_b) liền kề nhau bên trong từ w .

Cặp (t_a, t_b) có $freq$ lớn nhất sẽ được ghép lại thành một token mới. Quy trình này giúp cân bằng giữa từ vựng kích thước cố định và khả năng biểu diễn từ chưa biết (Out-Of-Vocabulary).

Vietnamese Tokenization (Tách từ tiếng Việt với Underthesea)

Đối với tiếng Việt, do đặc thù ngôn ngữ đơn lập, các âm tiết đứng rời rạc nhưng thường kết hợp tạo thành từ ghép hoặc từ láy mang ngữ nghĩa chặt chẽ. Việc sử dụng các công cụ Tokenization đa ngữ (như BPE) cắt theo khoảng trắng có thể phá vỡ hoàn toàn cấu trúc từ vựng lõi. Để khắc phục, hệ thống tích hợp thư viện NLP **Underthesea** nhằm thực hiện tách bạch ranh giới từ (Word Segmentation) trước khi đưa vào biểu diễn Vector.

Sự cần thiết của tokenizer đặc thù được thể hiện rõ qua ví dụ kinh điển sau đây với câu: *‘Học sinh học sinh học’*.

- **1. Phân tách đa ngữ thông thường (chia theo âm tiết khoảng trắng):**

→ [‘Học’, ‘sinh’, ‘học’, ‘sinh’, ‘học’]

Cách chia này làm mất hoàn toàn phân định ngữ nghĩa ban đầu của câu, khiến

mô hình nhằm lẫn giữa chủ thể và cụm danh từ.

• 2. Tách từ tiếng Việt chuyên dụng với Underthesea:

→ [‘Học_sinh”, ‘học”, ‘sinh_học”]

Hai từ ghép ‘*học sinh*” (danh từ chỉ người) và ‘*sinh học*” (danh từ chỉ môn học) được phát hiện liền mạch và nối nối với nhau chuẩn xác bằng dấu gạch dưới, giúp mô hình đọc hiểu đúng đại ý gốc một cách dễ dàng.

Quá trình tách từ đóng vai trò tiền xử lý cực kỳ quan trọng, cung cấp đầu vào chất lượng cao và bảo toàn trọn vẹn mạch ý nghĩa của văn bản tiếng Việt.

0.1.2 Word/Document Embeddings (Biểu diễn Vector)

Embeddings là cách máy tính biểu diễn token hoặc toàn bộ văn bản dưới dạng vector số học dày đặc (Dense Vector) gồm các số thực liên tục nhằm lưu giữ đặc trưng ngữ nghĩa. Các từ có nghĩa tương đồng sẽ nằm gần nhau hơn trong một không gian vector đa chiều (ví dụ 512, 768 chiều). Để đo mức độ tương đồng giữa hai vector mô tả ngữ nghĩa, ta sử dụng công thức **Cosine Similarity (Độ tương đồng Cosin)**:

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}} \quad (2)$$

Trong đó:

- $\mathbf{A}, \mathbf{B}$: Là hai vector đặc trưng nhiều chiều (n-chiều) biểu diễn cho hai văn bản.
- A_i, B_i : Thành phần thứ i của vector $\mathbf{A}$ và $\mathbf{B}$.
- $\|\mathbf{A}\|, \|\mathbf{B}\|$: Độ dài (norm L2) của mỗi vector.

Giá trị cosin nằm trong khoảng $[-1, 1]$, trong đó 1 chỉ ra hai vector cùng hướng (ý nghĩa giống nhau hoàn toàn), 0 là trực giao (không liên quan), và -1 là ngược hướng.

0.1.3 Kiến trúc Transformer và Cơ chế Attention

Kể từ khi ra mắt vào năm 2017, kiến trúc Transformer đã trở thành sợi chỉ đỏ của NLP hiện đại. Thay vì xử lý tuần tự (như RNN/LSTM), Transformer xử lý văn bản song song thông qua cơ chế Tự chú ý (Self-Attention). Cơ chế **Scaled Dot-Product Attention** cho phép mô hình đánh giá tầm quan trọng của các từ trong câu khi đang xét một từ bất kỳ. Công thức lõi của cơ chế này là:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (3)$$

Trong đó:

- Q (Query - Truy vấn): Vector biểu diễn từ đang cần tìm độ tương quan phân tích (thứ ta đi tìm).
- K (Key - Khóa): Vector biểu diễn để đối chiếu độ tương hợp với Query cho mọi từ trong câu (thứ ta kiểm tra).
- V (Value - Giá trị): Vector mang nội dung ngữ nghĩa thực sự của từ đó sẽ được giữ lại.
- d_k : Chiều không gian của vector Key, việc chia cho $\sqrt{d_k}$ giúp thu nhỏ phương sai của Tích vô hướng (dot product) tránh việc hàm softmax rơi vào vùng bão hòa gradient (gradient vanishing).
- Hàm softmax đảm bảo ma trận trọng số luôn có phạm vi $[0, 1]$ và tổng phân bố là 1.

0.2 Mô hình ngôn ngữ lớn (Large Language Models - LLM)

Khái quát về thành phần tạo nên "bộ não" suy luận của hệ thống.

0.2.1 Tổng quan và Bản chất dự đoán

Sự tiến hóa từ các mô hình biểu diễn ngữ nghĩa thành các LLM có khả năng suy luận mạnh mẽ là kết quả của việc gia tăng kích thước mạng nơ-ron và dữ liệu sinh văn bản (Causal Language Modeling). Bài toán cốt lõi của LLM là **Dự đoán token tiếp theo (Next-token Prediction)** dựa trên chuỗi từ đã có. Xác suất tạo ra một câu chữ chiều dài T được định nghĩa theo chuỗi phân phối xác suất có điều kiện:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^T P(w_t \mid w_1, \dots, w_{t-1}) \quad (4)$$

Trong đó:

- w_t : Token mục tiêu tại vị trí t .
- $w_1, \dots, w_{t-1}$: Các token tiền cảnh (context) đã được cung cấp (từ prompt sinh ra phía trước).

Mô hình sẽ chọn token tiếp theo thông qua việc lấy mẫu (Sampling) theo hàm phân phối xác suất này.

0.2.2 Kỹ thuật Prompt Engineering

Là quá trình thiết kế, điều chỉnh các chỉ thị đầu vào (prompt) nhằm định hướng LLM sinh ra câu trả lời theo ý muốn mà không phải đào tạo lại trọng số. Một số kỹ thuật cốt lõi:

- **Zero-shot**: Yêu cầu trả lời trực tiếp mà không cung cấp mẫu trước.

- **Few-shot learning:** Cung cấp một vài ví dụ input-output để mô hình nội suy quy luật.
- **Chain-of-Thought (CoT):** Chỉ thị mô hình suy luận từng bước thay vì nhảy ngay ra kết quả cuối cùng (giúp giảm thiểu sai sót toán học/logic).

0.2.3 Khả năng gọi hàm (Function Calling / Tool Use)

Đây là đặc tính quan trọng giúp LLM kết nối với thế giới bên ngoài thay vì chỉ sinh text bị cô lập. Khi cấu hình một prompt đi kèm danh sách các "Tool", mô hình khi cần dữ liệu hoặc thực hiện thao tác sẽ tự động trả về chuỗi văn bản dạng cấu trúc (JSON) mô tả tên hàm và danh sách tham số (Arguments) thay vì sinh chuỗi text tự do.

0.2.4 Hiện tượng Hallucination (Ảo giác) trong LLM

Bản chất của LLM là dự đoán chuỗi từ mang ý nghĩa tự nhiên nhất, chúng không "nhớ" thông tin theo các bản ghi chính xác. Vì thế, khi đối diện với thông tin chuyên ngành mới hoặc tự suy đoán sự kiện, LLM gặp ảo giác — sinh ra thông tin thoát nhìn rất đáng tin cậy nhưng sai hoàn toàn sự thật. Nguyên nhân cốt lõi do dữ liệu đào tạo (out-of-date) và sự thiếu vắng nền tảng bám trụ kiến thức thực tế (lack of grounding). Điều này là căn cứ mạnh mẽ để đưa công nghệ RAG vào hệ thống.

0.3 Retrieval-Augmented Generation (RAG)

Thành phần quan trọng nhất giúp hệ thống trả lời chính xác dựa trên Sách giáo khoa trực tiếp.

0.3.1 Kiến trúc RAG cơ bản

Nguyên lý hoạt động cơ bản của RAG gồm 3 bước tuần tự: Truy xuất (Retrieval) - Bổ sung (Augmented) - Sinh văn bản (Generation).

- **Pha Offline (Chuẩn bị dữ liệu):** Tập hợp tài liệu nội dung tài liệu, phân mảnh nhỏ (chunking), sau đó quy đổi mảnh bằng Embedding Model và lưu trữ vào Vector Database.
- **Pha Online (Truy vấn):** Nhận câu hỏi thực, mã hóa vector câu hỏi đưa đi tìm kiếm chuỗi nội dung giống nhất (Retrieval), đưa đoạn văn bản kết quả thu được bổ sung vào Prompt hiện tại (Augmented) rồi đưa vào tính toán trả kết quả bởi LLM (Generation).

0.3.2 Data Chunking (Phân mảnh dữ liệu)

Do các LLM có hạn chế về kích cỡ cửa sổ khung ngữ cảnh (Context Window), nguyên văn tài liệu bắt buộc phải được chia nhỏ (Chunking). Các chiến lược chia nhỏ thường gặp:

- **Fixed-size Chunking:** Cắt nội dung theo chuỗi số lượng token nhất định liên tiếp, có dải đè (overlap) để nối mạch văn.
- **Recursive Character Chunking:** Cắt theo phân cấp ngắt đoạn lô-gic như đoạn văn ($\backslash n \backslash n$), câu xuống dòng ($\backslash n$), khoảng trắng.
- **Hierarchical Chunking (Phân mảnh phân cấp):** Rạch rời dựa trên cấu trúc chương/mục nhằm gắn kết Metadata để duy trì cây tri thức vững chắc mà không làm đứt gãy luồng thông tin văn bản.

0.3.3 Vector Database (Cơ sở dữ liệu Vector)

Nơi lưu trữ Vector Embeddings và hỗ trợ việc tìm kiếm đánh giá mức độ tương đồng giữa không gian các vector. Điểm vượt trội cốt lõi của cơ sở dữ liệu Vector so với các hệ quản trị CSDL quan hệ truyền thống là khả năng tính toán ưu việt chuẩn xác khoảng cách và tọa độ (ví dụ *Euclidean*, *Dot-Product*, *Cosine*) giữa Vector Truy vấn (Query Vector) và hàng triệu Vector gốc của các mảnh tài liệu (Document Vectors), nhằm trả về nhanh chóng các ngữ nghĩa đối sánh nhất.

0.4 Kỹ thuật truy xuất thông tin nâng cao (Advanced Retrieval)

Đi sâu vào các thuật toán tối ưu hóa việc tìm kiếm tài liệu từ kho Sách giáo khoa (Corpus).

0.4.1 Lexical Search (Tìm kiếm từ khóa)

Thuật toán cơ sở dùng để tra cứu sự tương đồng phần chữ cái đối với từ khóa hiếm/chuyên ngành. Nổi bật là cơ chế **TF-IDF** (Term Frequency - Inverse Document Frequency). **Công thức TF-IDF cơ bản:**

$$TF_IDF(t, d, D) = TF(t, d) \times IDF(t, D) \quad (5)$$

Trong đó:

- $TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$: Tần suất (mật độ) từ t xuất hiện trong văn bản d .
- $IDF(t, D) = \log \frac{N}{|\{d \in D: t \in d\}|}$: Cơ chế đo lường mức độ đặc trưng chuyên biệt với việc nghịch đảo số tài liệu liên quan (N là tổng số văn bản).

Tuy nhiên, TF-IDF không có điểm trần và dễ bị thiên lệch chiều dài văn bản. Hiện tại phiên bản tiến tiến hơn được sử dụng là **BM25 (Best Matching 25)** với giới hạn tần suất:

$$Score_{BM25}(Q, d) = \sum_{q_i \in Q} IDF(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)} \quad (6)$$

Trong đó:

- Q : Câu truy vấn bao gồm các từ khóa q_i .
- $f(q_i, d)$: Số lần xuất hiện từ khóa q_i trong văn bản d .
- $|d|$ và $avgl$: Chiều dài độ dài văn bản d và trung bình cộng độ dài hệ thống.
- k_1 và b : Các siêu tham số tinh chỉnh độ dốc phạt do độ dài văn bản và trần bảo hòa của tần suất điểm BM25.

0.4.2 Semantic Search (Tìm kiếm ngữ nghĩa)

Thay vì tra xét từ khóa theo dạng văn bản bề mặt, *Semantic Search* đánh giá sự tương đồng trực tiếp về mặt ý nghĩa. Cả truy vấn q và văn bản d trong hệ thống đều được mô hình nhúng ánh xạ thành các dense vector V_q và V_d trong không gian n chiều. Sự tương đồng ngữ nghĩa sau đó được xác định thông qua các độ đo khoảng cách hình học cốt lõi.

Hai biến thể phương trình phổ biến nhất trong hệ thống Vector Search:

1. Tích vô hướng (Dot Product) Đo lường trực tiếp độ lớn và mức độ đồng hướng của hai vector:

$$Score_{dot}(q, d) = V_q \cdot V_d = \sum_{i=1}^n V_{q,i} V_{d,i} \quad (7)$$

Phép đo này tính toán cực kỳ nhanh và phù hợp khi các vector đã được chuẩn hóa độ dài (normalized).

2. Độ tương đồng Cosine (Cosine Similarity)

Độ tương đồng Cosine được sử dụng để đo lường góc giữa vector truy vấn và vector tài liệu, qua đó giảm ảnh hưởng của độ dài văn bản và chỉ tập trung vào mức độ tương đồng về mặt ngữ nghĩa giữa các vector.

Điểm số xếp hạng của tài liệu lúc này được xác định trực tiếp thông qua giá trị của hàm Cosine Similarity, như đã được trình bày tại **Công thức 2**.

Đoạn tài liệu nào tạo ra độ tương đồng cao nhất với vector truy vấn theo một trong hai thang đo trên sẽ được hệ thống Vector Search ưu tiên trích xuất và chuyển sang giai đoạn xếp hạng tiếp theo.

0.4.3 Hybrid Search (Tìm kiếm lai) và thuật toán RRF

Mục đích kết hợp sự chính xác ngữ pháp vững từ BM25 và sự nhận thức khái niệm từ Vector Search (Semantic). **Reciprocal Rank Fusion (RRF)** là một thuật toán tích hợp ưu điểm của bảng xếp thứ hạng của hai chiến lược tìm kiếm. Cách

tính điểm RRF bù trừ của một đoạn tìm kiếm kết quả d :

$$Score_{RRF}(d) = \frac{1}{k + rank_{BM25}(d)} + \frac{1}{k + rank_{Vector}(d)} \quad (8)$$

Trong đó:

- $rank_{BM25}(d)$, $rank_{Vector}(d)$: Trật tự xếp hạng của văn bản d truy được dựa theo từng cơ chế.
- k : Hằng số giảm xóc (thường từ 60).

0.4.4 Query Optimization (Tối ưu hóa truy vấn)

Tiền xử lý dùng nội tính của LLM nâng cao khả năng câu hỏi:

- **Query Rewriting (Viết lại truy vấn)**: Chuyển câu hỏi thô thành các keywords chuyên biệt bám bản thông tin tài liệu.
- **HyDE (Hypothetical Document Embeddings)**: Yêu cầu mô hình tưởng tượng (hallucinate) nội suy nhanh giả câu trả lời, sử dụng đoạn đó làm khung Vector đi tìm phần tham chiếu sát thực tiễn để tăng phần cover (độ phủ tìm kiếm).

0.4.5 Reranking (Xếp hạng lại)

Thay Bi-Encoder bằng việc dùng **Cross-Encoder** đánh bằng điểm Top 10-20 về lại. Bi-Encoder Embedding lưu rõ hai chuỗi làm chậm chạp thiếu nhạy bén trong ngữ cảnh. Khối Cross-Encoder cho trượt luôn cặp chuỗi ký tự Text gốc vào một lớp Self-Attention làm cho sự đánh giá trở nên cực kỳ tinh xác (tất nhiên chi phí truy vấn thời gian nhỉnh hơn bởi đối sánh chéo đôi).

0.5 Multi-Agent Systems (Hệ thống đa tác tử)

Lý thuyết phục vụ cho tính năng thiết kế Bài giảng, Slide hoặc cấu trúc Giáo án tự động.

0.5.1 AI Agent (Tác tử AI)

Triển khai môi trường nơi mà Agent lấy tư duy từ lõi LLM kèm hệ Tools (Công cụ tương tác được ra API hay code bên ngoài – ví dụ đọc code, gọi python tính, trích xuất wiki).

0.5.2 Khung lý thuyết ReAct (Reasoning and Acting)

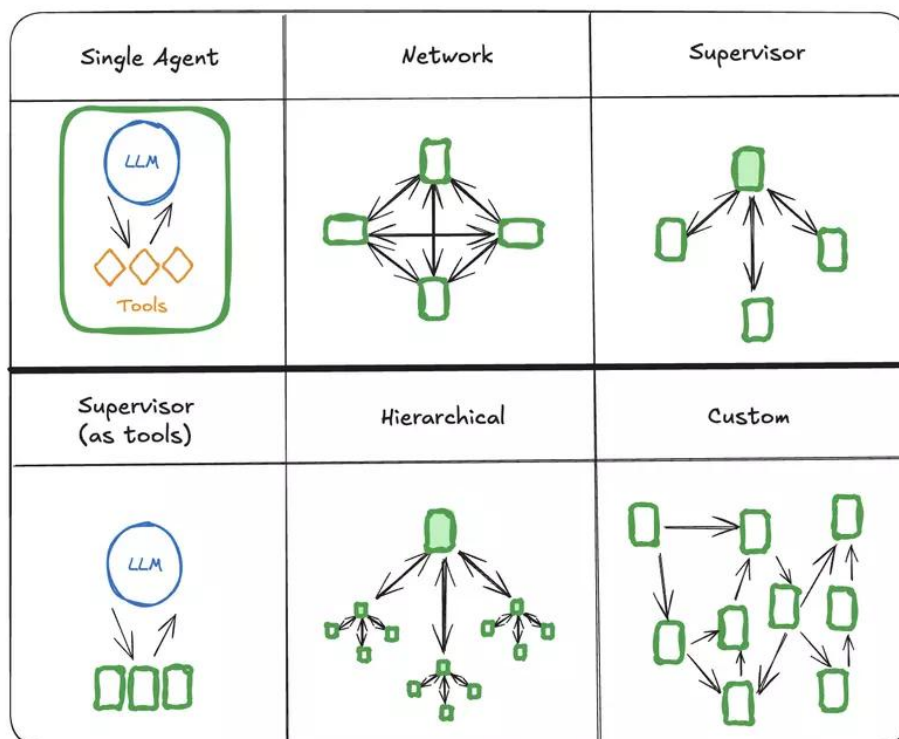
Một vòng lặp giải bài toán chia tuần tự:

- **Thought (Tư duy)**: Nhận xét phương hướng tương ứng tiếp diễn.
- **Action (Hành động)**: Điều động chạy tham số gọi Tool cung cấp.
- **Observation (Quan sát)**: Kịch bản quan sát phản hồi kết xuất rồi chạy vòng

ngữ Thought để ra trả lời kết.

0.5.3 Các kiến trúc Đa tác tử (Multi-Agent Architectures)

Thay vì phụ thuộc vào một mô hình ngôn ngữ duy nhất để thực hiện toàn bộ tác vụ từ A đến Z, hệ thống đa tác tử (Multi-Agent) chia nhỏ bài toán phức tạp thành nhiều phần và giao cho từng tác tử chuyên biệt xử lý. Việc tổ chức các tác tử này thành một kiến trúc luồng dữ liệu hợp lý là chìa khóa để vận hành hệ thống trơn tru.



Hình 0.1: Tổng quan các mô hình tổ chức trong hệ thống Đa tác tử (Multi-Agent)

Như được trình bày ở Hình 0.1, có rất nhiều kiểu kiến trúc điều phối mạng lưới AI Agent khác nhau, bao gồm:

- **Mô hình mạng lưới (Network):** Các tác tử giao tiếp tự do với nhau không qua trung gian. Cấu trúc này linh hoạt nhưng dễ rối loạn luồng thông tin khi hệ thống phình to.
- **Mô hình hội đồng (Joint / Committee):** Các tác tử cùng ngồi lại báo cáo và thảo luận giải quyết cùng một vấn đề.
- **Mô hình dạng chuỗi (Chain / Sequential):** Output của tác tử này là Input của tác tử kế tiếp, theo một đường truyền cứng.
- **Mô hình Giám sát (Supervisor / Hierarchical):** Một tác tử đóng vai trò nhạc trưởng (Nhà quản lý) đứng ở vị trí trung tâm, chịu trách nhiệm nhận yêu cầu,

phân loại và gọi các tác tử chuyên biệt bên dưới thực thi.

Mô hình Supervisor (Điều phối tác tử) và lựa chọn kiến trúc

Trong đồ án này, kiến trúc **Supervisor** được chọn làm thiết kế lõi cho hệ thống trợ lý học tập. Một Supervisor Agent sẽ tiếp nhận câu hỏi, phân tích ý định thay vì xử lý trực tiếp, sau đó định tuyến đến các Agent chuyên sâu tương ứng (VD: Tìm lý thuyết, Sinh bài tập...).

Việc lựa chọn này mang lại 3 lợi thế cốt lõi: (1) **Kiểm soát luồng chặt chẽ**, giúp hệ thống suy luận minh bạch, tránh LLM lạc đề hoặc vòng lặp vô hạn; (2) **Tính đóng gói & phân quyền**, giới hạn quyền truy cập tài nguyên CSDL rõ ràng cho từng Agent; (3) **Dễ dàng mở rộng (Scalability)**, cho phép nhanh chóng thêm các Agent chức năng mới sau này mà không làm xáo trộn kiến trúc tổng thể.

0.5.4 Cơ chế thực thi song song (Fan-out / Fan-in)

Cách chia song song. (Phối cử sub-agent tìm Mở Đầu độc lập với sub-agent tìm Bài Tập) và sau đó tổng kết dồn thông tin (Fan-in) quy tụ làm tiết kiệm mạnh chi phí chờ.

0.6 Framework Đánh giá hệ thống RAG (RAG Evaluation)

Cách đo lường và định lượng bằng số (Metrics) độ chính xác của hệ thống AI bằng các framework chuẩn (ví dụ RAGAS).

0.6.1 Mô hình LLM-as-a-Judge

Sử dụng LLM đóng vai giám khảo cho việc tính hệ số chính xác không can thiệp sức người.

0.6.2 Bộ khung đo lường RAG (RAG Metrics)

Để định lượng hiệu năng của hệ thống RAG, các framework hiện đại (như RAGAS) chia việc đánh giá thành 4 thang đo độc lập, giúp kiểm tra cả chất lượng truy xuất tài liệu lẫn năng lực sinh văn bản của LLM:

- **Độ trung thực (Faithfulness):** Đảm bảo mọi luận điểm (claims) có trong câu trả lời sinh ra đều có thể được chứng minh bởi tài liệu ngữ cảnh (context) truy xuất được, chống lại hiện tượng ảo giác (hallucination).

$$Faithfulness = \frac{|V \cap C|}{|V|} \quad (9)$$

Trong đó, V là tập các luận điểm rút ra từ câu trả lời của LLM, và C là tập các luận điểm có thể suy diễn trực tiếp từ ngữ cảnh.

- **Mức độ bám sát câu hỏi (Answer Relevance):** Đo lường mức độ câu trả lời

giải quyết trực tiếp vấn đề được đặt ra trong câu hỏi của người dùng, không thừa thãi hay đi lệch trọng tâm.

$$Answer\ Relevance = \frac{1}{N} \sum_{i=1}^N \cos(E_q, E_{g_i}) \quad (10)$$

Trong đó, từ câu trả lời, tiến hành sinh ngược ra N câu hỏi giả định g_i . Sau đó tính trung bình độ tương đồng Cosine giữa vector nhúng của câu hỏi gốc E_q và các câu hỏi giả định E_{g_i} .

- **Độ bao phủ câu trả lời (Context Recall):** Đo lường việc hệ thống tìm kiếm có lấy về đủ mọi thông tin cần thiết để giải quyết câu hỏi mẫu (Ground Truth) hay không.

$$Context\ Recall = \frac{|GT \cap R|}{|GT|} \quad (11)$$

Trong đó, GT là tập thông tin thiết yếu có trong đáp án mẫu (Ground Truth), và R là tập thông tin được tìm thấy trong ngữ cảnh được truy xuất (Retrieved Context).

- **Độ chính xác truy xuất (Context Precision):** Đánh giá khả năng xếp hạng (ranking) của hệ thống tìm kiếm, xem những tài liệu chứa thông tin hữu ích nhất có được xếp ở các vị trí đầu tiên hay không.

$$Context\ Precision = \frac{\sum_{k=1}^K (Precision@k \times v_k)}{\text{Tổng số tài liệu liên quan trong } K} \quad (12)$$

Trong đó, K là tổng số tài liệu được truy xuất, $v_k = 1$ nếu tài liệu ở vị trí k có chứa đáp án và $v_k = 0$ nếu ngược lại. $Precision@k$ là độ chính xác tính đến vị trí thứ k .